โจทย์ที่ 1: สถานการณ์ที่ 1 - ระบบ Flash Sale

สถานการณ์

บริษัทกำลังจะจัดแคมเปญ **"Flash Sale"** แจกโค้ดส่วนลด **1,000 สิทธิ์** โดยจะเปิดให้กดพร้อมกันเวลา 12:00 น.

คาดว่าจะมีผู้ใช้งาน (Traffic) เข้ามาพร้อมกันบนระบบทั้ง Web และ Mobile App ประมาณ **50,000 คน** ภายใน 5 นาทีแรก

คำถาม

ในฐานะ SA ให้ออกแบบ High-Level System Architecture สำหรับระบบนี้

1. คุณจะใช้ Tech Stack หรือ Component ใดบ้าง (เช่น Database, Cache, Message Queue ฯลฯ) เพื่อให้ระบบไม่ล่ม?
2. คุณจะออกแบบ Logic อย่างไรเพื่อแก้ปัญหา Concurrency (Race Condition) เพื่อป้องกันไม่ให้คนกดรับโค้ดส่วนลดซ้ำซ้อนหรือแจกโค้ดเกิน 1,000 สิทธิ์?

คำตอบ

Tech Stack และ Component ที่ใช้

Component	Tech Stack	หน้าที่หลัก
Edge & Traffic Control	Cloudflare	แคชหน้า Static ไว้ที่ Edge 100% เพื่อไม่ให้เซิร์ฟเวอร์เสีย CPU ส่งหน้าเว็บ และตั้ง Rate Limit กัน Bot
API Gateway	Nginx Ingress	ทำ Per-User Rate Limiting (1 User ID กดได้ไม่เกิน 1 ครั้งต่อวินาที) และตรวจสอบ JWT Signature
Application Service	Node.js	รองรับ Concurrent Connections สูงและกิน Memory น้อย ตั้งค่า Pre-scale Pods รอไว้ล่วงหน้าได้
In-Memory Cache	Redis Cluster	ใช้เช็กและตัดสิทธิ์จบใน RAM รวดเร็ว
Message Broker	RabbitMQ	ทำหน้าที่เป็น Shock Absorber รับข้อมูลเฉพาะ 1,000 คนที่ได้สิทธิ์ ส่งต่อไปบันทึกลง DB ตามจังหวะที่ DB รับไหว
Database	MySQL (InnoDB)	เก็บข้อมูล Transaction โดยรับโหลดจาก Worker

การออกแบบ Logic แก้ปัญหา Concurrency (Race Condition)

สำหรับการแก้ปัญหา Race Condition ในฝั่ง **Node.js** คู่กับ **Redis**:

- **ห้าม** เขียนโค้ดแบบสั่ง `await redis.get()` มาเช็กค่าใน if แล้วค่อยสั่งหักลบแยกบรรทัด
- ใช้วิธี **Atomic Operation** แทน

1. ป้องกันการกดซ้ำ (Idempotency)

ใช้ `redis.sadd()` แทนการเช็กแบบแยกคำสั่ง:

```
const added = await redis.sadd('claimed_users', userId);
if (added === 0) {
  // ผู้ใช้นี้เคยรับสิทธิ์ไปแล้ว
  return { success: false, message: 'คุณรับสิทธิ์ไปแล้ว' };
}
```

- ถ้ากดครั้งแรก → Redis จะคืนค่า **1** → ให้ดำเนินการต่อ
- ถ้ากดซ้ำ (หลายแท็บ / สคริปต์ยิงรัว) → Redis จะคืนค่า **0** → ตอบกลับทันทีว่า "คุณรับสิทธิ์ไปแล้ว"

2. ตัดโควตา 1,000 สิทธิ์

ใช้ `redis.decr()` ซึ่งเป็นคำสั่ง Atomic:

```
const remaining = await redis.decr('flash_sale_quota');

if (remaining >= 0) {
  // ยังมีสิทธิ์เหลือ (ได้เป็น 1 ใน 1,000 คนแรก)
  await queue.publish('claim_success', { userId });
} else {
  // สิทธิ์เต็มแล้ว (คนที่ 1,001 เป็นต้นไป)
  await redis.incr('flash_sale_quota'); // Rollback
  isSoldOut = true;
  return { success: false, message: 'สิทธิ์เต็มแล้ว' };
}
```

ข้อดีของวิธีนี้:

- DECR จะเรียงคิวหักลบทีละคนโดยอัตโนมัติ
- ได้เลขตั้งแต่ 999 ลงมาถึง 0 → ได้สิทธิ์
- ได้เลขติดลบ (-1) → สิทธิ์เต็มแล้ว

3. รวมขั้นตอนด้วย Node.js

ใช้โค้ด Node.js ร่วมกับคำสั่ง Atomic ของ Redis โดยตรง เพื่อรวมการเช็กชื่อและหักโควตา:

```
async function claimFlashSale(userId) {
  // 1. ป้องกันกดซ้ำด้วย SADD (Atomic)
  const added = await redis.sadd('claimed_users', userId);
  if (added === 0) {
    return { success: false, message: 'คุณรับสิทธิ์ไปแล้ว' };
  }

  // 2. หักโควตาด้วย DECR (Atomic)
  const remaining = await redis.decr('flash_sale_quota');

  if (remaining >= 0) {
    // ได้สิทธิ์ → ส่งเข้า Queue
    await queue.publish('claim_success', { userId });
    return { success: true, message: 'รับสิทธิ์สำเร็จ' };
  } else {
    // สิทธิ์เต็ม → Rollback ทั้งโควตาและชื่อ
    await redis.incr('flash_sale_quota');
    await redis.srem('claimed_users', userId);
    isSoldOut = true;
    return { success: false, message: 'สิทธิ์เต็มแล้ว' };
  }
}
```

ข้อดีของวิธี Node.js นี้:

- ใช้คำสั่ง Atomic ของ Redis (`SADD` , `DECR` , `INCR` , `SREM`) ซึ่งปลอดภัยจาก Race Condition
- เขียนและดูแลง่าย
- Rollback ได้ครบทั้งชื่อผู้ใช้และโควตาเมื่อสิทธิ์เต็ม

> **สรุป:** ระบบนี้ใช้ Redis เป็นจุดตัดสิทธิ์หลัก (เร็วและ Atomic) + RabbitMQ เป็นบัฟเฟอร์กันโหลดลง Database + Edge Cache และ Rate Limit กัน Traffic Spike ทำให้รองรับ 50,000 คนกดพร้อมกันได้โดยระบบไม่ล่ม และไม่มีการแจกโค้ดเกิน 1,000 สิทธิ์